

Computational Fluid Dynamics
MAE-6263
Spring 2018

FINAL PROJECT

2D Navier-Stokes Solver for Channel Flow using Collocated Grid

Saadbin Khan

**School of Mechanical & Aerospace Engineering
Oklahoma State University**

May 13, 2018

1. Introduction

The objective of this project is to solve 2-D Navier-Stokes equations for the classical pressure driven channel flow configuration. The main challenge is to attain pressure-velocity coupling and two different methods will be applied to address this challenge with a view to understanding the measures that need to be taken to overcome that. The method differs in the treatment of pressure Poisson equation only. We used two iterative schemes to deal with it: one without momentum interpolation, and the other with momentum interpolation. A simple compact stencil is used solve for pressure which demonstrates gradual increment of divergence of the solution field and it can be avoided with momentum interpolation. The time integration scheme that we used is of explicit forward Euler manner.

2. Problem Statement

The rectangular 2D channel under current study is illustrated below:

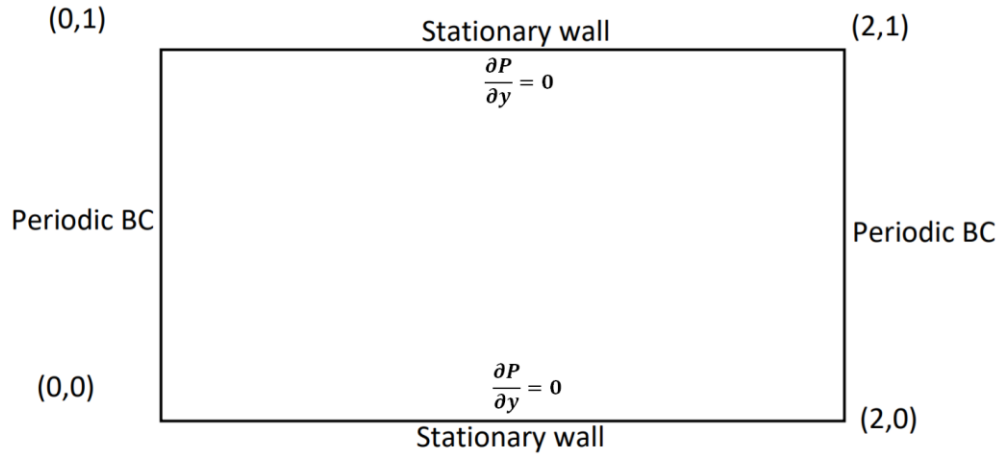


Figure 1: The schematic of the 2D channel based on the given boundary conditions

The domain to be solved for is a rectangular domain of unit width and length of two units.

The initial condition for velocity is given by:

$$\text{At } t = 0 \quad \vec{u}(x, y) = 0 \text{ for } 0 \leq x \leq 2, 0 \leq y \leq 1; \quad (1)$$

The initial condition for pressure is given by:

$$\text{At } t = 0 \quad p(x, y) = 0 \text{ everywhere except boundaries}; \quad (2)$$

Bououndary condition for velocity is given by:

$$BC : \left\{ \begin{array}{l} \vec{u}(x, y) = 1 \text{ for } y = 1; \\ \vec{u}(x, y) = 0 \text{ for } y = 0; \\ \vec{u}(x, y) = \vec{u}(x - 2, y) \text{ for } x = 2; \\ \vec{u}(x, y) = \vec{u}(x + 2, y) \text{ for } x = 0; \end{array} \right\} \quad (3)$$

Bououndary condition for velocity is given by:

$$BC : \left\{ \begin{array}{l} p(x, y) = p_{outlet} \text{ for } x = 2; \\ p(x, y) = p_{inlet} \text{ for } x = 0; \\ \frac{\partial p}{\partial t} = 0 \text{ for } y = 0 \text{ and } y = 1; \end{array} \right\} \quad (4)$$

3. Mathematical Formulation and Code Development

3.1 2D Navier Stokes Equation

Over the domain Ω the flow is governed by 2D Navier-Stokes equation. The generalized vector form of the incompressible NS equation is given by:

$$\frac{\partial \vec{u}}{\partial t} + \vec{u} \cdot \nabla \vec{u} = -\frac{\nabla p}{\rho} + \nu \Delta \vec{u} \quad (5)$$

$$\nabla \cdot \vec{u} = 0 \text{ (Continuity)} \quad (6)$$

Where, $\frac{\partial \vec{u}}{\partial t}$ = Unsteady term

$\vec{u} \cdot \nabla \vec{u}$ = Advective term

$\nu \Delta \vec{u}$ = Diffusion term

$\frac{\nabla p}{\rho}$ = Pressure term

3.2 Pressure Poisson Equation

Pressure Poisson equation is needed to solve to attain the velocity-pressure coupling in incompressible flow solution. To derive the equation we need to start from the 2D Navier Stokes equation (eqn. 5) and take a divergence of both side of the equation:

$$\nabla \cdot \left(\frac{\partial \vec{u}}{\partial t} + \vec{u} \cdot \nabla \vec{u} \right) = \nabla \cdot \left(-\frac{\nabla p}{\rho} + \nu \Delta \vec{u} \right) \quad (7)$$

$$\text{Or, } \frac{\partial (\nabla \cdot \vec{u})}{\partial t} = \nabla \cdot (-\vec{u} \cdot \nabla \vec{u} + \nu \Delta \vec{u}) + \nabla \cdot \left(-\frac{\nabla p}{\rho} \right) \quad (8)$$

From continuity we know that $\nabla \cdot \vec{u} = 0$ and we define

$$\bar{H} = -\vec{u} \cdot \nabla \vec{u} + \nu \Delta \vec{u} \quad (9)$$

So for constant ρ we get:

$$\frac{\partial (\nabla \cdot \vec{u})}{\partial t} = \nabla \cdot \bar{H}$$
$$\nabla \cdot \nabla p = \rho \nabla \cdot \bar{H}$$

So, we now got out pressure Poisson equation in continuous domain:

$$\nabla^2 p = \rho \nabla \cdot \bar{H} \quad (10)$$

3.3 Explicit Solution Algorithm

Using equation (9) into equation (5) and assuming incompressible flow with unit density we get:

$$\frac{\partial \vec{u}}{\partial t} = \bar{H} - \nabla p \quad (11)$$

We use first order forward Euler for time integration. The discretization for (n+1)th time level is as following:

$$\frac{\vec{u}^{n+1} - \vec{u}^n}{\Delta t} = \bar{H}^n - \nabla p^n \quad (12)$$

So, we develop a explicit solution pseudocode as following:

Pseudo Code for Explicit Solution
• Set: Initial & Boundary values for $\vec{u}$ and p • Set tolerance and divergence_int • While(divergence > tolerance) • Compute $\bar{H} = -\vec{u} \cdot \nabla \vec{u} + \nu \Delta \vec{u}$ • Initialize GS_l1 norm • While(GS_l1norm > tolerance) ▪ Solve pressure Poisson: $\nabla^2 p = \rho \nabla \cdot \bar{H}$ • End with updated p • Compute: $\vec{u}^{n+1} = \vec{u}^n + \Delta t(\bar{H}^n - \nabla p^n)$ • Compute divergence • end

Figure 2: Pseudo Code for Explicit Solution

3.4 Conservation of Kinetic Energy

For both gradient and divergence we used central difference scheme for both. Kinetic energy conservation is a consequence of discretized momentum equation and the equation can be derived by taking the scalar product of velocity times momentum equation:

$$[Discretized u_i - momentum equation] \times u_i \Rightarrow u_i G_i p \Delta \Omega$$

Energy conservation requires:

$$\sum_{i=1}^N u_i G_i p \Delta \Omega = \sum_{S_b} p \mathbf{v}_n \Delta S - \sum_N p D_i u_i \Delta \Omega \quad (13)$$

This implies the following which needs to hold true if energy conservation is to be confirmed:

$$\sum_{i=1}^N (u_i G_i p + p D_i u_i) \Delta \Omega = \text{surface terms} \quad (14)$$

To check we take 4 nodal points from 1 to 4 in an 1D domain ($\Delta \Omega = 1$), that mean we put $N = 4$ in equation (14) and we get:

$$\sum_{i=1}^N (u_i G_i p + p D_i u_i) \Delta \Omega = \sum_{i=1}^N u_i (p_{i+1} - p_{i-1}) + p_i (u_{i+1} - u_{i-1}) \quad (15)$$

$$\begin{aligned}
&= u_1 p_2 - u_1 p_0 + p_1 u_2 - p_1 u_0 + u_2 p_3 - u_2 p_1 + p_2 u_3 - p_2 u_1 \\
&\quad + u_3 p_4 - u_3 p_2 + p_3 u_4 - p_3 u_2 + u_4 p_5 - u_4 p_3 + p_4 u_5 - p_4 u_3 \\
&= u_1 p_2 - u_1 p_0 + p_1 u_2 - p_1 u_0 + u_2 p_3 - u_2 p_1 + p_2 u_3 - p_2 u_1 \\
&\quad + u_3 p_4 - u_3 p_2 + p_3 u_4 - p_3 u_2 + u_4 p_5 - u_4 p_3 + p_4 u_5 - p_4 u_3 \\
&= -u_1 p_0 - p_1 u_0 + u_4 p_5 + p_4 u_5 = \text{surface terms}
\end{aligned}$$

So, it is evident that the choice of gradient and divergence schemes for this project fulfills the conservation of energy criteria.

3.5 Compact Scheme with 2D Finite Difference

3.5.1 Scheme without Momentum Interpolation in PPE

We need to discretize each term in equation (12) using central difference scheme for all the divergence and gradient and first order upwind to estimate the phase velocities:

$$u_e = u_{i+\frac{1}{2},j}, \quad u_w = u_{i-\frac{1}{2},j}, \quad u_n = u_{i,j+\frac{1}{2}} \text{ and } u_s = u_{i,j-\frac{1}{2}}$$

We define ∇p^n to be ΔG_p^n assuming $G_p = \Delta p$, so, the discretized form for H and G terms become:

$$\bar{H}_{i+\frac{1}{2},j}^n = \frac{1}{2}(\bar{H}_{i+1,j}^n + \bar{H}_{i,j}^n) \quad (16.1)$$

$$\bar{H}_{i-\frac{1}{2},j}^n = \frac{1}{2}(\bar{H}_{i,j}^n + \bar{H}_{i-1,j}^n) \quad (16.2)$$

$$\bar{H}_{i,j+\frac{1}{2}}^n = \frac{1}{2}(\bar{H}_{i,j+1}^n + \bar{H}_{i,j}^n) \quad (16.3)$$

$$\bar{H}_{i,j-\frac{1}{2}}^n = \frac{1}{2}(\bar{H}_{i,j}^n + \bar{H}_{i,j-1}^n) \quad (16.4)$$

$$G_{p_{i+\frac{1}{2},j}}^n = \frac{1}{\Delta x}(p_{i+1,j}^n - p_{i,j}^n) \quad (16.5)$$

$$G_{p_{i-\frac{1}{2},j}}^n = \frac{1}{\Delta x}(p_{i,j}^n - p_{i-1,j}^n) \quad (16.6)$$

$$G_{p_{i,j+\frac{1}{2}}}^n = \frac{1}{\Delta y}(p_{i,j+1}^n - p_{i,j}^n) \quad (16.7)$$

$$G_{p_{i,j-\frac{1}{2}}}^n = \frac{1}{\Delta y}(p_{i,j}^n - p_{i,j-1}^n) \quad (16.8)$$

The overall pressure Poisson equation (10) discretized equation comes down to:

$$\frac{G_{p_{i+\frac{1}{2},j}}^n - G_{p_{i-\frac{1}{2},j}}^n}{\Delta x} + \frac{G_{p_{i,j+\frac{1}{2}}}^n - G_{p_{i,j-\frac{1}{2}}}^n}{\Delta y} = \frac{\bar{H}_{i+\frac{1}{2},j}^n - \bar{H}_{i-\frac{1}{2},j}^n}{\Delta x} + \frac{\bar{H}_{i,j+\frac{1}{2}}^n - \bar{H}_{i,j-\frac{1}{2}}^n}{\Delta y} \quad (17)$$

Then, using the decomposition in equations (16.1-16.8) we get:

$$\frac{\frac{1}{\Delta x}(p_{i+1,j}^n - p_{i,j}^n) - \frac{1}{\Delta x}(p_{i,j}^n - p_{i-1,j}^n)}{\Delta x} + \frac{\frac{1}{\Delta y}(p_{i,j+1}^n - p_{i,j}^n) - \frac{1}{\Delta y}(p_{i,j}^n - p_{i,j-1}^n)}{\Delta y} = \frac{\frac{1}{2}(\bar{H}_{i+1,j}^n + \bar{H}_{i,j}^n) - \frac{1}{2}(\bar{H}_{i,j}^n + \bar{H}_{i-1,j}^n)}{\Delta x} + \frac{\frac{1}{2}(\bar{H}_{i,j+1}^n + \bar{H}_{i,j}^n) - \frac{1}{2}(\bar{H}_{i,j}^n + \bar{H}_{i,j-1}^n)}{\Delta y} \quad (18)$$

Simplifying:

$$\frac{p_{i+1,j}^n - 2p_{i,j}^n + p_{i-1,j}^n}{\Delta x^2} + \frac{p_{i,j+1}^n - 2p_{i,j}^n + p_{i,j-1}^n}{\Delta y^2} = \frac{\bar{H}_{i+1,j}^n - \bar{H}_{i-1,j}^n}{2\Delta x} + \frac{\bar{H}_{i,j+1}^n - \bar{H}_{i,j-1}^n}{2\Delta y} \quad (19)$$

This is our final discretized scheme for pressure Poisson equation. After this we just need to update the velocity field with solved pressure:

$$\bar{u}_{i,j}^{n+1} = \bar{u}_{i,j}^n + \Delta t (\bar{H}_{i,j}^n - \Delta p_{i,j}^n) \quad (20)$$

The convergence criterion is set as the L1-norm to be less than 10^{-4} :

$$\sum_{i=1, j=1}^{i=N, j=N} |u_{i,j}^{k+1} - u_{i,j}^k| + |v_{i,j}^{k+1} - v_{i,j}^k| < 10^{-4} \quad (21)$$

This scheme ensures strong coupling between velocity and pressure. But, the problem with the method is that the conservation of mass is failed. This is because, the treatment of pressure gradient in PPE and momentum equation is not consistent. In PPE, the gradient is calculated directly without any interpolation. On the other hand in momentum equation it is calculated with interpolation. This leads to the divergence of the updated velocity field to be non-zero:

$$\Delta \cdot \bar{u}_{i,j}^{n+1} = \varepsilon \neq 0 \text{ (or machine precision)}$$

This ε accumulates over time and gives rise to convergence error. Additional measures need to be taken to satisfy mass conservation. The next section of the report discusses momentum interpolation method which is a way to get around this problem.

3.5.2 Scheme with Momentum Interpolation in PPE

Momentum interpolation is one of the ways to avoid the ‘leakage’ or ‘accumulation’ of mass, which occurs due to the fact that the solution acquired from the above scheme was not divergence-free due to inconsistency in the treatment of pressure. For this method, everything else other than the discretization of PPE will be similar. The same sort of interpolation that we do in the momentum equation’s pressure gradient needs to be applied here for face velocities around the node that we are calculating in. For example, for $u_{i+\frac{1}{2},j}$ the interpolation is applied like following:

$$u_{i+\frac{1}{2},j} = \frac{1}{2}(u_{i,j} + u_{i+1,j}) + \frac{1}{2} \left[\left(\frac{\Delta \Omega_i}{A_i^t} + \frac{\Delta \Omega_{i+1}}{A_{i+1}^t} \right) (p_i - p_{i+1}) - \frac{\Delta \Omega_i}{A_i^t} (p_{i+1} - p_{i-1}) - \frac{\Delta \Omega_{i+1}}{A_{i+1}^t} (p_{i+1} - p_{i-1}) \right] \quad (22)$$

The second term in this expression is the pressure correction term. As we are using uniformly spaced grid, so we can simplify this to:

$$u_{i+\frac{1}{2},j} = \frac{1}{2}(u_{i,j} + u_{i+1,j}) + \frac{\Delta \Omega}{4A_p} (p_{i+2,j} - 3p_{i+1,j} + 3p_{i,j} - p_{i-1,j}) \quad (23.1)$$

Similarly for other faces necessary:

$$u_{i-\frac{1}{2},j} = \frac{1}{2}(u_{i,j} + u_{i+1,j}) + \frac{\Delta \Omega}{4A_p} (p_{i+1,j} - 3p_{i,j} + 3p_{i-1,j} - p_{i-2,j}) \quad (23.2)$$

$$v_{i,j+\frac{1}{2}} = \frac{1}{2}(v_{i,j} + v_{i,j+1}) + \frac{\Delta \Omega}{4A_p} (p_{i,j+2} - 3p_{i,j+1} + 3p_{i,j} - p_{i,j-1}) \quad (23.3)$$

$$v_{i,j-\frac{1}{2}} = \frac{1}{2}(v_{i,j} + v_{i,j+1}) + \frac{\Delta \Omega}{4A_p} (p_{i,j+1} - 3p_{i,j} + 3p_{i,j-1} - p_{i,j-2}) \quad (23.4)$$

Now, we apply the following for continuity:

$$\frac{u_{i+\frac{1}{2},j} - u_{i-\frac{1}{2},j}}{\Delta x} + \frac{v_{i,j+\frac{1}{2}} - v_{i,j-\frac{1}{2}}}{\Delta y} = 0 \quad (24)$$

This equation (24) is used as the momentum interpolated pressure Poisson equation. The velocity field update maintains the same process as the without momentum interpolation method.

3.6 Designing Numerical Experiment

Given information about some of the numerical parameters are:

$$\begin{aligned} \text{Reynolds number, } Re &= \frac{V_{peak} L_y}{\nu} = 100 \\ \text{Viscosity, } \nu &= 1.0 \times 10^{-2} \text{ cm}^2/\text{s} \\ \text{Density, } \rho &= 1 \text{ gm/cc} \\ \text{Peclet number, } Pe_{\Delta x} &= \frac{V_{peak} \Delta x}{\nu} = 1.5 \end{aligned}$$

So from this information and considering uniform grid $\Delta x = \Delta y$ we find:

$$\text{Peak centerline velocity, } V_{peak} = 1 \text{ cm/s}$$

The analytic expression for the parabolic velocity profile for a 2D channel flow between stationary parallel plates is given by:

$$u(y) = \frac{Ky^2}{2} + 1 \quad (24)$$

Where,

$$\frac{\partial^2 u}{\partial y^2} = \frac{f'(x)}{\vartheta} = K \quad (25)$$

$$\frac{\partial p}{\partial x} = f'(x) = \vartheta \frac{\partial^2 u}{\partial y^2} \quad (26)$$

Now if we mark the centerline to be $y = 0$ and upper and lower wall to be $y = h = \pm 0.5$, then we integrate it over y to get the mass flow rate per unit transverse area:

$$\dot{m} = \frac{1}{2h} \times 2 \times \int_0^h \left(1 - \frac{y^2}{h^2}\right) dy = \frac{2}{3} \quad (27)$$

Now,

$$\dot{m} = \rho A u \quad (28)$$

As $\rho = 1$ and $A = 1$ (unit traverse area), so:

$$u_{mean} = \frac{2}{3} \quad (29)$$

Moreover,

$$K = -\frac{2}{h^2} = -8 \quad (30)$$

Now, form equation (26)

$$\frac{\partial p}{\partial x} = \vartheta K = -0.08 \quad (31)$$

Integrating equation (31) over x directional domain, we get:

$$P_{outlet} - P_{inlet} = -0.08(1 - 0)$$

So, the final equation for pressure difference is:

$$P_{inlet} = P_{outlet} + 0.16 \quad (32)$$

In our problem $P_{outlet} = 100.0$, so,

$$P_{inlet} = 100.16$$

4. Numerical Analysis

Two different simulations have been performed: one without momentum interpolation and the other one with momentum interpolation. Following are two subsections containing results for each simulation. Each subsection contains: iso-contours and pressure field in steady state, pressure and velocity distribution in y , change of mass flow rate and energy with time.

4.1 Experiment without Momentum Interpolation in PPE

4.1.1 Isocontour and Pressure

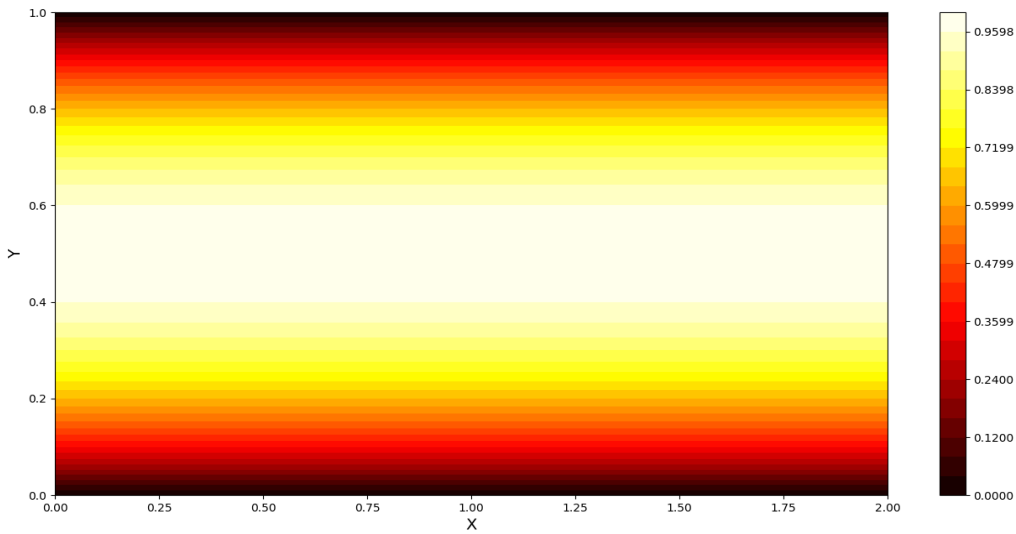


Figure 1: Velocity iso-contour at steady state

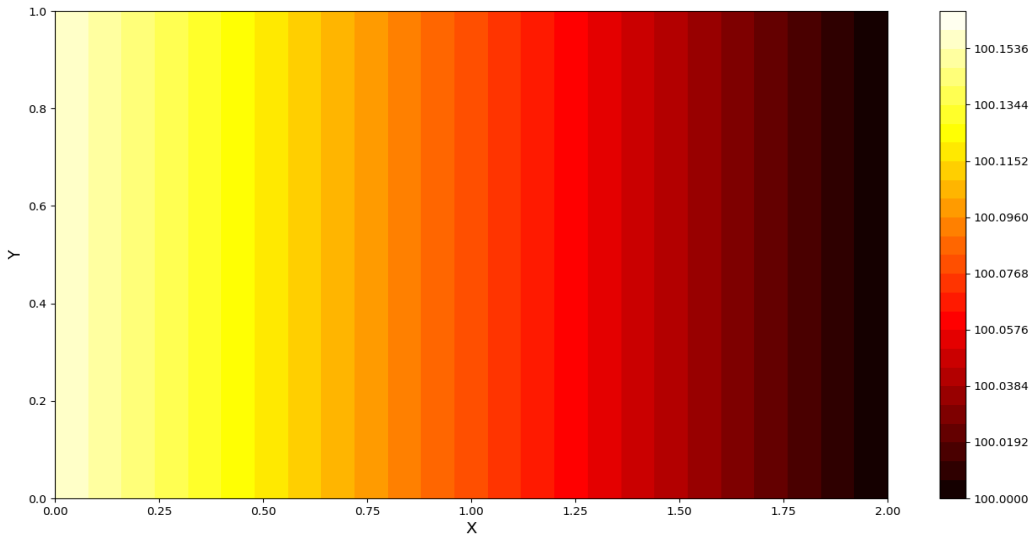


Figure 2: Pressure iso-contour at steady state

4.1.2 Pressure and Velocity Profiles

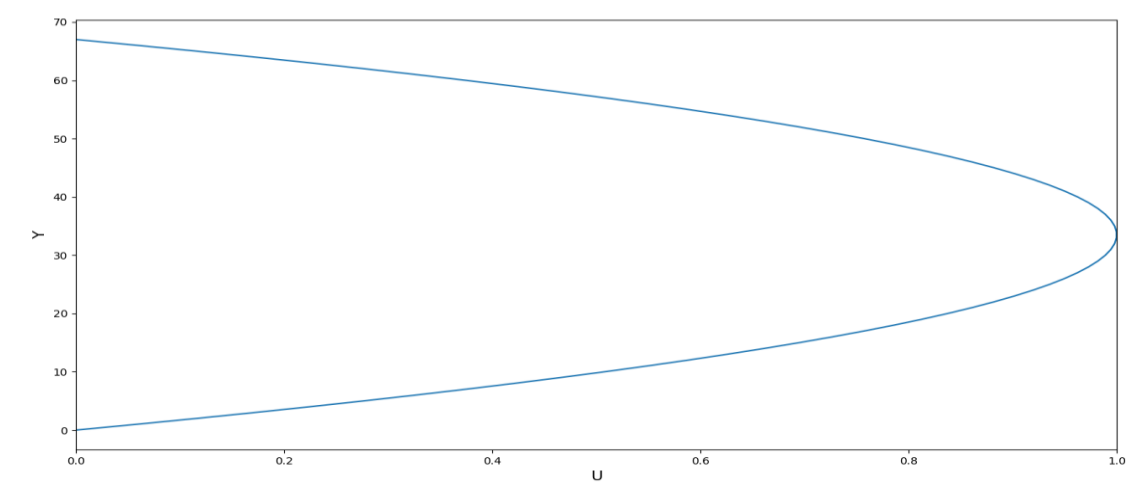


Figure 3: Velocity profile with Y for U at mid-x at steady state

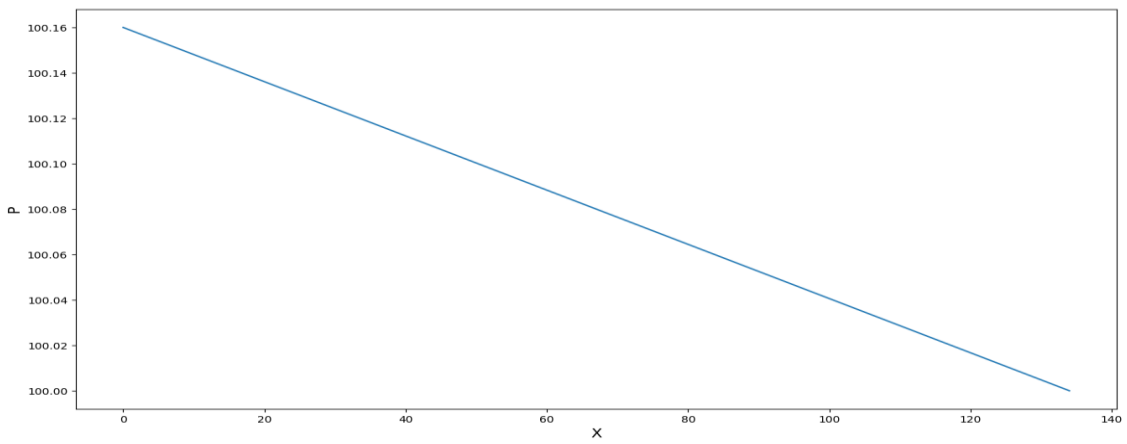


Figure 4: Pressure profile with Y at mid-x at steady state

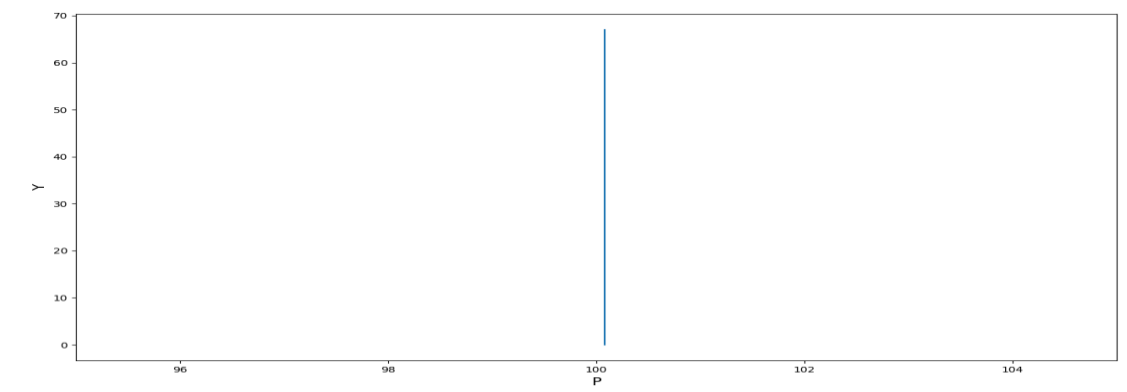


Figure 5: Pressure profile with X at mid-y at steady state

4.1.3 Mass flow rate

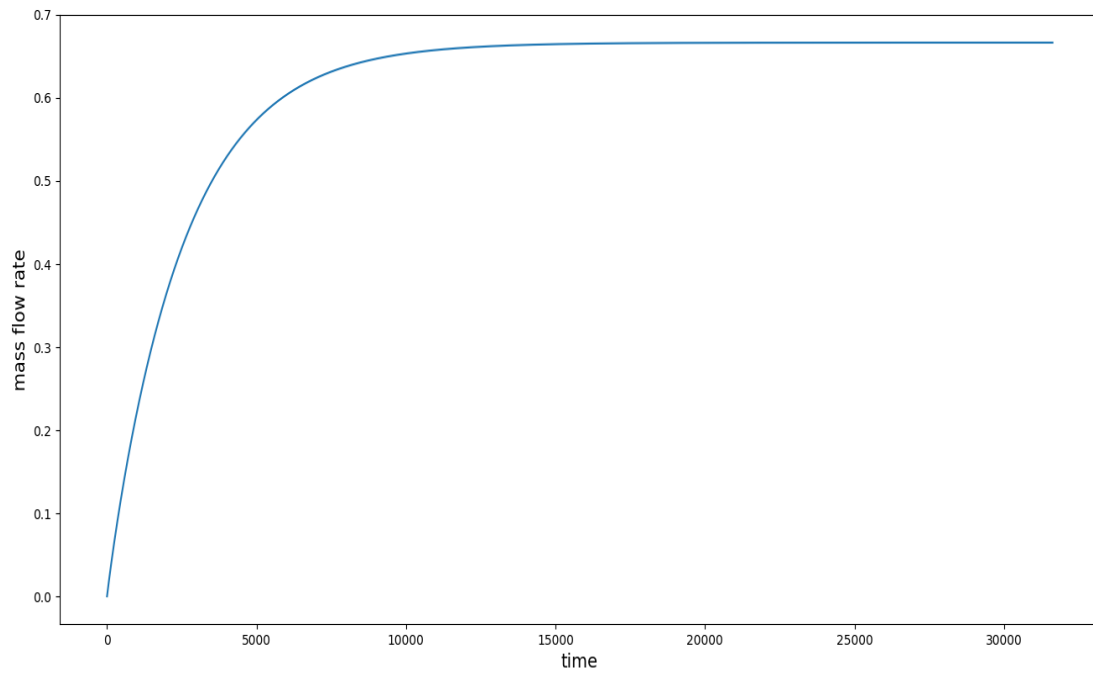


Figure 6: Mass flow rate with time upto steady state

4.1.4 Energy

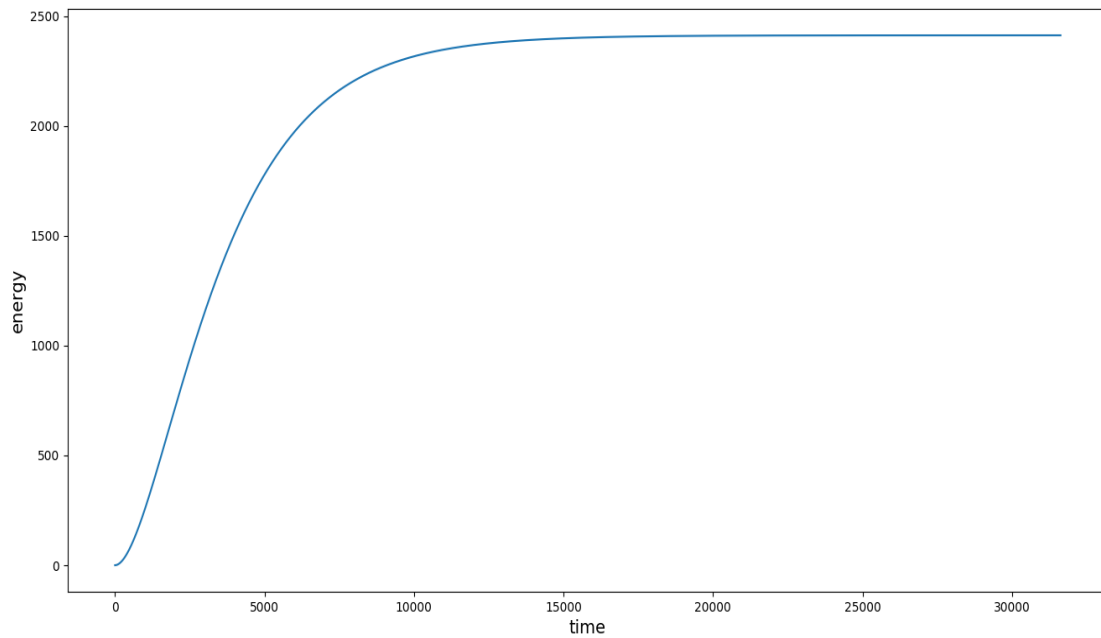


Figure 7: Energy with time upto steady state

4.1.5 Discussion

- Figure 1. Shows uniform velocity distribution in x direction. It is evident from the contour that U-velocity varies only in y direction. Near the wall the velocity is minimum (=0) due to no-slip condition and near the centerline it is highest ($U_{peak} = 1.0$)
- Figure 2. Shows pressure distribution to be constant in y direction, but variable from 100.00 to 100.16 in x direction.
- Figure 3. Is the velocity profile at mid-x which shows peak velocity to be 1.0 at the centerline.
- Figure 4. Shows linear drop of pressure along x axis
- Figure 5. Shows that along y, there is no change of pressure
- Figure 6. is plotted from the following equation for energy:

$$\sum_{j=0}^{ny} \sum_{i=0}^{nx} \rho \frac{u_{i,j}^2}{2} \quad (33)$$

Energy reaches a steady state value when steady state flow condition is achieved which is ≈ 2400

- Figure 7. is plotted from the following equation for mass flow rate:

$$\frac{\int_1^{Ny} u dy}{L_y} \quad (34)$$

Mass flow rate reaches a steady state value when steady state flow condition is achieved which is ≈ 0.6665 . This is consistent with equation (27) where we analytically calculated the mass flow rate to be $\frac{2}{3}$

4.2 Experiment with Momentum Interpolation in PPE

The code is attached in the appendix. It was taking a long time to come down inside tolerance level and the convergence became exponentially slower with time. So I could not plot anything for this case.

APPENDIX

Without Momentum Interpolation

```
!!!!!!!!!!!!!!!!!!!!!! 2D Navier Stokes Equation !!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!! Channel Flow in Collocated Grid !!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!! Without Momentum Interpolation !!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!! SAADBIN KHAN !!!!!!!!!!!!!!!!!!!!!!!
```

```
program ns_2d
```

```
implicit none
```

```
real(8),parameter :: re=100.0, pe=1.5, gam=0.2, nu=0.01, ro=1.0, tol=0.00001
integer(8),parameter :: timestep=10000,nscheme=5 ! needed?
```

```
real(8),parameter :: xs=0.0,xe=2.0,ys=0.0,ye=1.0
integer(8) :: nx,ny,nstep,n,i,j,k
real(8) :: dx,dy,dt,tmax,llns,lln,tol2,tol_t,mass,energy
real(8) :: t1,t2,t3,t4,start,finish,upeak,rhs,lhs
real(8),allocatable ::
u(:,,:),v(:,,:),e(:,,:),p(:,,:),p1(:,,:),un(:,,:),uold(:,,:),vold(:,,:),vn(:,,:),x(:,),y(
:)
real(8),allocatable ::
uxx(:,,:),vxx(:,,:),uyy(:,,:),vyy(:,,:),ux(:,,:),vx(:,,:),uy(:,,:),vy(:,,:),grpx(:,,:),g
rpy(:,,)
real(8),allocatable :: hx(:,,:),hy(:,,:),hxx(:,,:),hyy(:,,)
integer(4) :: dig1, dig2, dig3, dig4,dig5,dig6
character(6) :: chits
```

```
upeak=re*nu/(ye-ys)
dx=pe*nu/upeak !0.04!
dy=dx
nx=int((xe-xs)/dx+1) !50
ny=int((ye-ys)/dy+1) !25
dx=(xe-xs)/nx
dy=(ye-ys)/ny
dt=gam*(40.0/45.0)*(dx**2)/nu !0.005
allocate(x(0:nx),y(0:ny))
```

```
do i=0,nx
    x(i)=xs+dx*float(i)
end do
do j=0,ny
    y(j)=ys+dy*float(j)
end do
```

```
print *,nx,ny,dx,dy,dt
```

```
allocate(u(-1:nx+1,0:ny),v(-1:nx+1,0:ny))
allocate(p(0:nx,-1:ny+1),p1(0:nx,-1:ny+1))
allocate(un(-1:nx+1,0:ny),vn(-1:nx+1,0:ny))
allocate(ux(0:nx,0:ny),vx(0:nx,0:ny),uy(0:nx,0:ny),vy(0:nx,0:ny))
allocate(uxx(0:nx,0:ny),vxx(0:nx,0:ny),uyy(0:nx,0:ny),vyy(0:nx,0:ny))
```

```
allocate(hx(-1:nx+1,0:ny),hy(0:nx,0:ny),hxx(0:nx,0:ny),hyy(0:nx,0:ny))
allocate(grpx(0:nx,0:ny),grpy(0:nx,0:ny),e(0:nx,0:ny))
```

!!!!!!! Without Momentum Interpolation !!!!!!!

```
if (nscheme==5) then !!!!!
```

```
u=0
v=0
p=0
grpx=0
grpy=0
p(0,:) = 100.16
p(nx,:) = 100.0
un=0
vn=0
nstep=0
tol_t=1
```

```
do while(tol_t>tol) !!! Time loop
```

```
    call cpu_time(start)
```

```
    nstep=nstep+1
```

```
    tol_t=0
```

```
    u(-1,:)=u(nx-1,:)
```

```
    v(-1,:)=v(nx-1,:)
```

```
    u(nx+1,:)=u(1,:)
```

```
    v(nx+1,:)=v(1,:)
```

```
    un(-1,:)=un(nx-1,:)
```

```
    vn(-1,:)=vn(nx-1,:)
```

```
    un(nx+1,:)=un(1,:)
```

```
    vn(nx+1,:)=vn(1,:)
```

```
    l1ns=0.0
```

```
    k=k+1
```

```
do i=0,nx
```

```
    do j=1,ny-1
```

```
        if(u(i,j)>=0) then
```

```
            ux(i,j) = (u(i,j)-u(i-1,j))/dx
```

```
            vx(i,j) = (v(i,j)-v(i-1,j))/dx
```

```
        else if(u(i,j)<0) then
```

```
            ux(i,j) = (u(i+1,j)-u(i,j))/dx
```

```
            vx(i,j) = (v(i+1,j)-v(i,j))/dx
```

```
        end if
```

```
        if(v(i,j)>=0) then
```

```
            uy(i,j) = (u(i,j)-u(i,j-1))/dy
```

```
            vy(i,j) = (v(i,j)-v(i,j-1))/dy
```

```
        else if(v(i,j)<0) then
```

```
            uy(i,j) = (u(i,j+1)-u(i,j))/dy
```

```
            vy(i,j) = (v(i,j+1)-v(i,j))/dy
```

```
        end if
```

```
        uxx(i,j) = (u(i+1,j)-2*u(i,j)+u(i-1,j))/(dx**2)
```

```

1,j))/ (dx**2)
vxx(i,j) = (v(i+1,j)-2*v(i,j)+v(i-
uyy(i,j) = (u(i,j+1)-2*u(i,j)+u(i,j-1))/ (dy**2)
vyy(i,j) = (v(i,j+1)-2*v(i,j)+v(i,j-1))/ (dy**2)
end do
ux(i,0) = ux(i,1)
vx(i,0) = vx(i,1)
uy(i,0) = uy(i,1)
vy(i,0) = vy(i,1)
ux(i,ny) = ux(i,ny-1)
vx(i,ny) = vx(i,ny-1)
uy(i,ny) = uy(i,ny-1)
vy(i,ny) = vy(i,ny-1)
uxx(i,0) = uxx(i,1)
vxx(i,0) = vxx(i,1)
uyy(i,0) = uyy(i,1)
vyy(i,0) = vyy(i,1)
uxx(i,ny) = uxx(i,ny-1)
vxx(i,ny) = vxx(i,ny-1)
uyy(i,ny) = uyy(i,ny-1)
vyy(i,ny) = vyy(i,ny-1)
end do

do i=0,nx
do j=0,ny
hx(i,j) = -u(i,j)*ux(i,j)-
v(i,j)*uy(i,j)+1/Re*(uxx(i,j)+uyy(i,j))
hy(i,j) = -u(i,j)*vx(i,j)-
v(i,j)*vy(i,j)+1/Re*(vxx(i,j)+vyy(i,j))
end do
end do

do j = 0,ny
hx(-1,j) = hx(nx-1,j)
hx(nx+1,j) = hx(1,j)
end do

do i = 0,nx
do j = 1,ny-1
hxx(i,j) = (hx(i+1,j)-hx(i-1,j))/(2*dx)
hyy(i,j) = (hy(i,j+1)-hy(i,j-1))/(2*dy)
end do
hxx(i,0) = hxx(i,1)
hxx(i,ny) = hxx(i,ny-1)
hyy(i,0) = hyy(i,1)
hyy(i,ny) = hyy(i,ny-1)
end do

p(:, -1)=p(:, 1)
p(:, ny+1)=p(:, ny-1)

tol2=1.0
do while(tol2>tol)
p1=p
do i = 1,nx-1

```

```

do j = 0,ny
    lhs = (p(i+1,j)+p1(i-
1,j)+p(i,j+1)+p1(i,j-1))/(dx**2)
    rhs = hxx(i,j) + hyy(i,j)
    p1(i,j) = (lhs-rhs)*dx*dx/4
end do
p1(:, -1)=p1(:, 1)
p1(:, ny+1)=p1(:, ny-1)

end do
e(0:nx,0:ny) = abs(p1(0:nx,0:ny)-p(0:nx,0:ny))
tol2=sum(e)

p = p1
end do

do j=1,ny-1
    do i=1,nx-1
        grpx(i,j) = (p(i+1,j)-p(i-1,j))/(2*dx)
        grpy(i,j) = (p(i,j+1)-p(i,j-1))/(2*dy)
    end do
end do
do j=0,ny
    grpx(0,j) = grpx(1,j)
    grpx(nx,j) = grpx(nx-1,j)
end do

do i = 0,nx
    do j = 1,ny-1
        u(i,j) = u(i,j) + dt*(hx(i,j)-grpx(i,j))
        v(i,j) = v(i,j) + dt*(hy(i,j)-grpy(i,j))
    end do
end do

mass=0.0
do j = 0,ny-1
    mass = mass+0.5*(u(nx/2+1,j) + u(nx/2+1,j+1))
end do
print *, nstep, ':::::', finish-start, ':::::', tol_t
open (unit=61,
file='mass.txt',position='append',action='write') !!!!!!!!!!!!!!! File names
Need to be changed
write(61,*) nstep, mass*dy
close(61)

energy=0.0
do i = 0,nx
    do j = 1,ny-1
        energy= energy+ro*u(i,j)*u(i,j)/2
    end do
end do
open (unit=7,
file='energy.txt',position='append',action='write') !!!!!!!!!!!!!!! File names
Need to be changed
write(7,*) nstep, energy
close(7)

```

```

        tol_t=sum(abs(un(0:nx,0:ny)-u(0:nx,0:ny))+abs(vn(0:nx,0:ny)-
v(0:nx,0:ny)))
        un(0:nx,1:ny-1)=u(0:nx,1:ny-1)
        vn(0:nx,1:ny-1)=v(0:nx,1:ny-1)
        !print *,nstep,':',k, 11n
        call cpu_time(finish)

        if(mod(nstep,1000)==0 .or. tol_t>tol) then
        dig1 =      nstep/100000 + 48
        dig2 = ( nstep - 100000*( nstep/100000 ) )/10000 + 48
        dig3 = ( nstep - 10000*( nstep/10000 ) )/1000 + 48
        dig4 = ( nstep - 1000*( nstep/1000 ) )/100 + 48
        dig5 =      ( nstep - 100*( nstep/100 ) )/10 + 48
        dig6 =      ( nstep - 10*( nstep/10 ) )/1 + 48
        chits(1:6) =
char(dig1)//char(dig2)//char(dig3)//char(dig4)//char(dig5)//char(dig6)
        ! print *,'FINAL TIME 3:::::::::', nstep, finish-start
        open (unit=8, file='u_vel'//chits//'.txt') !!!!!!!!!!!!!!! File names
Need to be changed
        do j=0,ny
            write(8,*) (u(i,j), i=0,nx)
        end do
        close(8)
        open (unit=9, file='p_vel'//chits//'.txt') !!!!!!!!!!!!!!! File names
Need to be changed
        do j=0,ny
            write(9,*) (p(i,j), i=0,nx)
        end do
        close(9)
        end if
    end do  !! Time loop ends

end if

end program

```


With Momentum Interpolation

```
!!!!!!!!!!!!!!!!!!!!!! 2D Navier Stokes Equation !!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!! Channel Flow in Collocated Grid !!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!! With Momentum Interpolation !!!!!!!!!!!!!!!!!!!!!!!
!!!!!!!!!!!!!!!!!!!!!! SAADBIN KHAN !!!!!!!!!!!!!!!!!!!!!!!

program ns_2d

implicit none

real(8),parameter :: re=100.0, pe=1.5, gam=0.2, nu=0.01, ro=1.0,
tol=0.00001,tolp=0.1
integer(8),parameter :: timestep=10000,nscheme=5 ! needed?

real(8),parameter :: xs=0.0,xe=2.0,ys=0.0,ye=1.0
integer(8) :: nx,ny,nstep,ppstep,n,i,j,k
real(8) :: dx,dy,dt,tmax,llns,lln,tol2,tol_t
real(8) :: t1,t2,t3,t4,start,finish,upeak,rhsx,lhsx,rhsy,lhsy
real(8),allocatable ::
u(:,,:),v(:,,:),e(:,,:),p(:,,:),p1(:,,:),un(:,,:),uold(:,,:),vold(:,,:),vn(:,,:),x(:,),y(
:)
real(8),allocatable ::
uxx(:,,:),vxx(:,,:),uyy(:,,:),vyy(:,,:),ux(:,,:),vx(:,,:),uy(:,,:),vy(:,,:),grpx(:,,:),g
rpy(:,,)
real(8),allocatable :: hx(:,,:),hy(:,,:),hxx(:,,:),hyy(:,,)
integer(4) :: dig1, dig2, dig3, dig4,dig5,dig6
character(6) :: chits

upeak=re*nu/(ye-ys)
dx=pe*nu/upeak !0.04!
dy=dx
nx=int((xe-xs)/dx+1) !50
ny=int((ye-ys)/dy+1) !25
dx=(xe-xs)/nx
dy=(ye-ys)/ny
dt=gam*(40.0/45.0)*(dx**2)/nu !0.005
allocate(x(0:nx),y(0:ny))

do i=0,nx
    x(i)=xs+dx*float(i)
end do
do j=0,ny
    y(j)=ys+dy*float(j)
end do

print *,nx,ny,dx,dy,dt

allocate(u(-1:nx+1,1:ny+1),v(-1:nx+1,1:ny+1))
allocate(p(-1:nx+1,-2:ny+2),p1(-1:nx+1,-2:ny+2))
!allocate(p(0:nx,-1:ny+1),p1(0:nx,-1:ny+1))
allocate(un(-1:nx+1,1:ny+1),vn(-1:nx+1,1:ny+1))
allocate(ux(0:nx,0:ny),vx(0:nx,0:ny),uy(0:nx,0:ny),vy(0:nx,0:ny))
allocate(uxx(0:nx,0:ny),vxx(0:nx,0:ny),uyy(0:nx,0:ny),vyy(0:nx,0:ny))
allocate(hx(-1:nx+1,0:ny),hy(0:nx,0:ny),hxx(0:nx,0:ny),hyy(0:nx,0:ny))
```

```
allocate(grpx(0:nx,0:ny),grpy(0:nx,0:ny),e(0:nx,0:ny))
```

```
!!!!!!! Explicit !!!!!!!!
```

```
if (nscheme==5) then
```

```
  u=0
  v=0
  p=0
  grpx=0
  grpy=0
  p(0,:) = 100.16
  p(nx,:) = 100.0
  un=0
  vn=0
  nstep=0
  tol_t=1
```

```
  do while(tol_t>tol) !!! Time loop
```

```
    call cpu_time(start)
```

```
    nstep=nstep+1
```

```
    tol_t=0
```

```
    u(-1,:)=u(nx-1,:)
```

```
    v(-1,:)=v(nx-1,:)
```

```
    u(nx+1,:)=u(1,:)
```

```
    v(nx+1,:)=v(1,:)
```

```
    un(-1,:)=un(nx-1,:)
```

```
    vn(-1,:)=vn(nx-1,:)
```

```
    un(nx+1,:)=un(1,:)
```

```
    vn(nx+1,:)=vn(1,:)
```

```
    llns=0.0
```

```
    k=k+1
```

```
    do i=0,nx
```

```
      do j=1,ny-1
```

```
        if(u(i,j)>=0) then
```

```
          ux(i,j) = (u(i,j)-u(i-1,j))/dx
```

```
          vx(i,j) = (v(i,j)-v(i-1,j))/dx
```

```
        else if(u(i,j)<0) then
```

```
          ux(i,j) = (u(i+1,j)-u(i,j))/dx
```

```
          vx(i,j) = (v(i+1,j)-v(i,j))/dx
```

```
        end if
```

```
        if(v(i,j)>=0) then
```

```
          uy(i,j) = (u(i,j)-u(i,j-1))/dy
```

```
          vy(i,j) = (v(i,j)-v(i,j-1))/dy
```

```
        else if(v(i,j)<0) then
```

```
          uy(i,j) = (u(i,j+1)-u(i,j))/dy
```

```
          vy(i,j) = (v(i,j+1)-v(i,j))/dy
```

```
        end if
```

```
        uxx(i,j) = (u(i+1,j)-2*u(i,j)+u(i-1,j))/(dx**2)
```

```
        vxx(i,j) = (v(i+1,j)-2*v(i,j)+v(i-
```

```
1,j))/(dx**2)
```

```
        uyy(i,j) = (u(i,j+1)-2*u(i,j)+u(i,j-1))/(dy**2)
```

```
        vyy(i,j) = (v(i,j+1)-2*v(i,j)+v(i,j-1))/(dy**2)
```

```
      end do
```

```
      ux(i,0) = ux(i,1)
```

```
      vx(i,0) = vx(i,1)
```

```

        uy(i,0) = uy(i,1)
        vy(i,0) = vy(i,1)
        ux(i,ny) = ux(i,ny-1)
        vx(i,ny) = vx(i,ny-1)
        uy(i,ny) = uy(i,ny-1)
        vy(i,ny) = vy(i,ny-1)
        uxx(i,0) = uxx(i,1)
            vxx(i,0) = vxx(i,1)
        uyy(i,0) = uyy(i,1)
        vyy(i,0) = vyy(i,1)
        uxx(i,ny) = uxx(i,ny-1)
            vxx(i,ny) = vxx(i,ny-1)
        uyy(i,ny) = uyy(i,ny-1)
        vyy(i,ny) = vyy(i,ny-1)
    end do

    do i=0,nx
        do j=0,ny
            hx(i,j) = -u(i,j)*ux(i,j)-
v(i,j)*uy(i,j)+1/Re*(uxx(i,j)+uyy(i,j))
            hy(i,j) = -u(i,j)*vx(i,j)-
v(i,j)*vy(i,j)+1/Re*(vxx(i,j)+vyy(i,j))
        end do
    end do

    p(:, -1)=p(:,1)
    p(:, -2)=p(:,2)
    p(-1, :)=2*p(0, :)-p(1, :)
    p(:, ny+1)=p(:,ny-1)
    p(:, ny+2)=p(:,ny-2)

    tol2=0.0001
    ppstep=0
    do while(tol2>tol2)
        ppstep=ppstep+1
        p1=p
        do i = 1,nx-1
            do j = 0,ny
                lhsx = 2*(u(i-1,j)-u(i+1,j))*dx*dx/dt
                rhsx = 4*(p(i+1,j)+p1(i-1,j))-p(i+2,j)-p1(i-
2,j)

                lhsy = 0 !2*(u(i,j-1)-u(i,j+1))*dy*dy/dt
                rhsy = 0 !4*(p(i,j+1)+p1(i,j-1))-p(i,j+2)-
p1(i,j-2)

                p1(i,j) = (lhsx+rhsx+lhsy+rhsy)/6 !12
            end do
            p1(:, -1)=p1(:,1)
            p1(:, ny+1)=p1(:,ny-1)
            p1(:, -2)=p1(:,2)
            p1(:, ny+2)=p1(:,ny-2)
            p1(-1, :)=2*p1(0, :)-p(1, :)
            p1(nx+1, :)=2*p1(nx, :)-p(nx-1, :)
        end do
        e(0:nx,0:ny) = abs(p1(0:nx,0:ny)-p(0:nx,0:ny))
        tol2=sum(e)
    end while
end

```

```

        !if (nstep<10 .and. mod(ppstep,10000)==0) print
*,ppstep,tol2
        !if (nstep>=10) print *,ppstep,tol2
        !if (ppstep==50000) exit
        p = p1
    end do

    do j=1,ny-1
        do i=1,nx-1
            grpx(i,j) = (p(i+1,j)-p(i-1,j))/(2*dx)
            grpy(i,j) = (p(i,j+1)-p(i,j-1))/(2*dy)
        end do
    end do
    do j=0,ny
        grpx(0,j) = grpx(1,j)
        grpx(nx,j) = grpx(nx-1,j)
    end do

    do i = 0,nx
        do j = 1,ny-1
            u(i,j) = u(i,j) + dt*(hx(i,j)-grpx(i,j))
            v(i,j) = v(i,j) + dt*(hy(i,j)-grpy(i,j))
        end do
    end do
    tol_t=sum(abs(un(0:nx,0:ny)-u(0:nx,0:ny))+abs(vn(0:nx,0:ny)-
v(0:nx,0:ny)))
    un(0:nx,1:ny-1)=u(0:nx,1:ny-1)
    vn(0:nx,1:ny-1)=v(0:nx,1:ny-1)
    call cpu_time(finish)
    print *, nstep, k, 11n, ':::::', finish-start, ':::::', tol_t
    print *,nstep,'::::::::::',k, 11n
    !if (mod(nstep,6)==0)pause
    call cpu_time(finish)

    if(mod(nstep,1)==0) then
        dig1 = nstep/100000 + 48
        dig2 = ( nstep - 100000*( nstep/100000 ) )/10000 + 48
        dig3 = ( nstep - 10000*( nstep/10000 ) )/1000 + 48
        dig4 = ( nstep - 1000*( nstep/1000 ) )/100 + 48
        dig5 = ( nstep - 100*( nstep/100 ) )/10 + 48
        dig6 = ( nstep - 10*( nstep/10 ) )/1 + 48
        chits(1:6) =
char(dig1)//char(dig2)//char(dig3)//char(dig4)//char(dig5)//char(dig6)
        ! print *,'FINAL TIME 3:::::::::::', nstep, finish-start
        open (unit=5, file='u_vel'//chits//'.txt') !!!!!!!!!!!!!!! File names
Need to be changed
        do j=0,ny
            write(5,*) (u(i,j), i=0,nx)
        end do
        close(5)
    end if
    end do !! Time loop ends

end if
end program

```